```
# ============================================================
# AI REQUIREMENTS INDEX - COMPLETE DATA ANALYTICS + ML PROJECT
# ============================================================

# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
```

```
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

import warnings
warnings.filterwarnings("ignore")



# ============================================================
# 2. LOAD DATASET
# ============================================================

df = pd.read_csv("ai-requirements-index.csv")

print("Dataset Loaded Successfully!")
print("Shape:", df.shape)

display(df.head())



# ============================================================
# 3. BASIC INFORMATION
# ============================================================

print("\nRows:", df.shape[0])
print("Columns:", df.shape[1])

print("\nColumn Names:")
```

```
print(df.columns.tolist())

print("\nData Types:")
print(df.dtypes)

print("\nDataset Information:")
df.info()



# ============================================================
# 4. STATISTICAL SUMMARY
# ============================================================

print("\nStatistical Summary:")
display(df.describe())



# ============================================================
# 5. MISSING VALUE CHECK
# ============================================================

print("\nMissing Values:")
print(df.isnull().sum())

print("\nTotal Missing Values:", df.isnull().sum().sum())



# ============================================================
# 6. DUPLICATE CHECK
```

```
# ============================================================

print("\nDuplicate Rows:", df.duplicated().sum())



# ============================================================
# 7. DATA CLEANING
# ============================================================

# Remove duplicate rows
df = df.drop_duplicates()

# Convert date column
df["snapshot_date"] = pd.to_datetime(
    df["snapshot_date"],
    dayfirst=True,
    errors="coerce"
)

# Numeric columns
numeric_columns = [
    "listings_with_ai",
    "total_listings",
    "pct",
    "required_count"
]

# Convert numeric columns
for col in numeric_columns:
```

```
    df[col] = pd.to_numeric(df[col], errors="coerce")

# Remove rows with missing important values
df = df.dropna(
    subset=[
        "category",
        "seniority",
        "tier",
        "listings_with_ai",
        "total_listings",
        "pct",
        "required_count"
    ]
)

print("\nAfter Cleaning:")
print("Shape:", df.shape)
print("Remaining Missing Values:", df.isnull().sum().sum())



# ============================================================
# 8. DATE FEATURE ENGINEERING
# ============================================================

df["year"] = df["snapshot_date"].dt.year
df["month"] = df["snapshot_date"].dt.month
df["day"] = df["snapshot_date"].dt.day

display(df.head())
```

```
# ============================================================
# 9. UNIQUE VALUES
# ============================================================

print("\nUnique Values:")

for col in df.columns:
    print(f"{col}: {df[col].nunique()} unique values")



# ============================================================
# 10. HISTOGRAMS
# ============================================================

numeric_cols = [
    "listings_with_ai",
    "total_listings",
    "pct",
    "required_count"
]

for col in numeric_cols:

    plt.figure(figsize=(8, 5))

    plt.hist(df[col], bins=30)
```

```
    plt.title(f"Histogram of {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")

    plt.show()


# ============================================================
# 11. BOX PLOTS
# ============================================================

for col in numeric_cols:

    plt.figure(figsize=(8, 4))

    plt.boxplot(df[col])

    plt.title(f"Box Plot of {col}")
    plt.ylabel(col)

    plt.show()


# ============================================================
# 12. CATEGORY DISTRIBUTION
# ============================================================

plt.figure(figsize=(10, 5))
```

```
df["category"].value_counts().plot(kind="bar")

plt.title("Number of Records by Category")
plt.xlabel("Category")
plt.ylabel("Count")
plt.xticks(rotation=45)

plt.show()



# ============================================================
# 13. SENIORITY DISTRIBUTION
# ============================================================

plt.figure(figsize=(8, 5))

df["seniority"].value_counts().plot(kind="bar")

plt.title("Records by Seniority")
plt.xlabel("Seniority")
plt.ylabel("Count")

plt.show()



# ============================================================
# 14. AI REQUIREMENT BY CATEGORY
# ============================================================
```

```
category_pct = (
    df.groupby("category")["pct"]
    .mean()
    .sort_values()
)

plt.figure(figsize=(10, 5))

category_pct.plot(kind="bar")

plt.title("Average AI Requirement Percentage by Category")
plt.xlabel("Category")
plt.ylabel("Average AI Requirement (%)")
plt.xticks(rotation=45)

plt.show()



# ============================================================
# 15. AI REQUIREMENT BY SENIORITY
# ============================================================

seniority_pct = (
    df.groupby("seniority")["pct"]
    .mean()
    .sort_values()
)

plt.figure(figsize=(8, 5))
```

```
seniority_pct.plot(kind="bar")

plt.title("Average AI Requirement by Seniority")
plt.xlabel("Seniority")
plt.ylabel("Average Percentage")

plt.show()



# ============================================================
# 16. CORRELATION HEATMAP
# ============================================================

corr = df[numeric_cols].corr()

plt.figure(figsize=(8, 6))

plt.imshow(corr, interpolation="nearest")

plt.xticks(
    range(len(corr.columns)),
    corr.columns,
    rotation=45,
    ha="right"
)

plt.yticks(
    range(len(corr.columns)),
```

```
    corr.columns
)

plt.colorbar()

plt.title("Correlation Heatmap")

plt.tight_layout()

plt.show()



# ============================================================
# 17. LINEAR REGRESSION - DATA PREPARATION
# ============================================================

X = df[
    [
        "category",
        "seniority",
        "tier",
        "listings_with_ai",
        "total_listings",
        "required_count",
        "year",
        "month",
        "day"
    ]
]
```

```
y = df["pct"]

X_train_lr, X_test_lr, y_train_lr, y_test_lr = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

categorical_features = [
    "category",
    "seniority",
    "tier"
]

numeric_features = [
    "listings_with_ai",
    "total_listings",
    "required_count",
    "year",
    "month",
    "day"
]

preprocessor_lr = ColumnTransformer(
    transformers=[
        (
            "cat",
```

```
            OneHotEncoder(handle_unknown="ignore"),
            categorical_features
        ),
        (
            "num",
            "passthrough",
            numeric_features
        )
    ]
)


# ============================================================
# 18. LINEAR REGRESSION
# ============================================================

linear_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor_lr),
        ("model", LinearRegression())
    ]
)

linear_model.fit(X_train_lr, y_train_lr)

y_pred_lr = linear_model.predict(X_test_lr)

mae = mean_absolute_error(y_test_lr, y_pred_lr)
rmse = np.sqrt(mean_squared_error(y_test_lr, y_pred_lr))
```

```
r2 = r2_score(y_test_lr, y_pred_lr)

print("\n==============================")
print("LINEAR REGRESSION RESULTS")
print("==============================")

print("MAE:", mae)
print("RMSE:", rmse)
print("R² Score:", r2)



# ============================================================
# 19. ACTUAL VS PREDICTED GRAPH
# ============================================================

plt.figure(figsize=(8, 6))

plt.scatter(y_test_lr, y_pred_lr)

plt.xlabel("Actual Percentage")
plt.ylabel("Predicted Percentage")

plt.title("Linear Regression: Actual vs Predicted")

plt.show()



# ============================================================
# 20. CREATE CLASSIFICATION TARGET
```

```
# ============================================================

threshold = df["pct"].median()

df["ai_requirement_class"] = (
    df["pct"] >= threshold
).astype(int)

print("\nClassification Threshold:", threshold)

print("\nClass Distribution:")
print(df["ai_requirement_class"].value_counts())



# ============================================================
# 21. CLASSIFICATION DATA
# ============================================================

X = df[
    [
        "category",
        "seniority",
        "tier",
        "listings_with_ai",
        "total_listings",
        "required_count",
        "year",
        "month",
        "day"
```

```
    ]
]

y = df["ai_requirement_class"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

preprocessor_tree = ColumnTransformer(
    transformers=[
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_features
        ),
        (
            "num",
            "passthrough",
            numeric_features
        )
    ]
)
```

```
# ============================================================
# 22. DECISION TREE - GINI INDEX
# ============================================================

dt_gini = Pipeline(
    steps=[
        ("preprocessor", preprocessor_tree),
        (
            "model",
            DecisionTreeClassifier(
                criterion="gini",
                max_depth=5,
                random_state=42
            )
        )
    ]
)

dt_gini.fit(X_train, y_train)

y_pred_gini = dt_gini.predict(X_test)

gini_accuracy = accuracy_score(y_test, y_pred_gini)
gini_precision = precision_score(y_test, y_pred_gini)
gini_recall = recall_score(y_test, y_pred_gini)
gini_f1 = f1_score(y_test, y_pred_gini)

print("\n==============================")
print("DECISION TREE - GINI")
```

```
print("================================")

print("Accuracy:", gini_accuracy)
print("Precision:", gini_precision)
print("Recall:", gini_recall)
print("F1 Score:", gini_f1)



# ============================================================
# 23. DECISION TREE - ENTROPY
# ============================================================

dt_entropy = Pipeline(
    steps=[
        ("preprocessor", preprocessor_tree),
        (
            "model",
            DecisionTreeClassifier(
                criterion="entropy",
                max_depth=5,
                random_state=42
            )
        )
    ]
)

dt_entropy.fit(X_train, y_train)

y_pred_entropy = dt_entropy.predict(X_test)
```

```
entropy_accuracy = accuracy_score(y_test, y_pred_entropy)
entropy_precision = precision_score(y_test, y_pred_entropy)
entropy_recall = recall_score(y_test, y_pred_entropy)
entropy_f1 = f1_score(y_test, y_pred_entropy)

print("\n==============================")
print("DECISION TREE - ENTROPY")
print("==============================")

print("Accuracy:", entropy_accuracy)
print("Precision:", entropy_precision)
print("Recall:", entropy_recall)
print("F1 Score:", entropy_f1)



# ============================================================
# 24. GINI VS ENTROPY GRAPH
# ============================================================

models = ["Gini", "Entropy"]

scores = [
    gini_accuracy,
    entropy_accuracy
]

plt.figure(figsize=(7, 5))
```

```
plt.bar(models, scores)

plt.title("Decision Tree: Gini vs Entropy")
plt.ylabel("Accuracy")

plt.ylim(0, 1)

plt.show()



# ============================================================
# 25. DECISION TREE VISUALIZATION
# ============================================================

tree_model = dt_gini.named_steps["model"]

feature_names = (
    dt_gini
    .named_steps["preprocessor"]
    .get_feature_names_out()
)

plt.figure(figsize=(22, 12))

plot_tree(
    tree_model,
    feature_names=feature_names,
    class_names=["Low", "High"],
    filled=False,
```

```
    rounded=True,
    max_depth=3
)

plt.title("Decision Tree using Gini Index")

plt.show()



# ============================================================
# 26. RANDOM FOREST
# ============================================================

random_forest = Pipeline(
    steps=[
        ("preprocessor", preprocessor_tree),
        (
            "model",
            RandomForestClassifier(
                n_estimators=100,
                max_depth=8,
                random_state=42
            )
        )
    ]
)

random_forest.fit(X_train, y_train)
```

```
y_pred_rf = random_forest.predict(X_test)

rf_accuracy = accuracy_score(y_test, y_pred_rf)
rf_precision = precision_score(y_test, y_pred_rf)
rf_recall = recall_score(y_test, y_pred_rf)
rf_f1 = f1_score(y_test, y_pred_rf)

print("\n==============================")
print("RANDOM FOREST RESULTS")
print("==============================")

print("Accuracy:", rf_accuracy)
print("Precision:", rf_precision)
print("Recall:", rf_recall)
print("F1 Score:", rf_f1)



# ============================================================
# 27. CONFUSION MATRIX
# ============================================================

cm = confusion_matrix(y_test, y_pred_rf)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Low", "High"]
)

disp.plot()
```

```
plt.title("Random Forest Confusion Matrix")

plt.show()



# ============================================================
# 28. CLASSIFICATION REPORT
# ============================================================

print("\n==============================")
print("RANDOM FOREST CLASSIFICATION REPORT")
print("==============================")

print(
    classification_report(
        y_test,
        y_pred_rf,
        target_names=["Low", "High"]
    )
)



# ============================================================
# 29. MODEL COMPARISON TABLE
# ============================================================

results = pd.DataFrame({
    "Model": [
```

```
        "Decision Tree - Gini",
        "Decision Tree - Entropy",
        "Random Forest"
    ],

    "Accuracy": [
        gini_accuracy,
        entropy_accuracy,
        rf_accuracy
    ],

    "Precision": [
        gini_precision,
        entropy_precision,
        rf_precision
    ],

    "Recall": [
        gini_recall,
        entropy_recall,
        rf_recall
    ],

    "F1 Score": [
        gini_f1,
        entropy_f1,
        rf_f1
    ]
})
```

```
print("\n==============================")
print("MODEL COMPARISON")
print("==============================")

display(results)



# ============================================================
# 30. MODEL PERFORMANCE GRAPH
# ============================================================

results.set_index("Model")[
    [
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ]
].plot(
    kind="bar",
    figsize=(12, 6)
)

plt.title("Machine Learning Model Performance")
plt.ylabel("Score")
plt.ylim(0, 1)

plt.xticks(rotation=20)
```

```
plt.legend()

plt.show()



# ============================================================
# 31. RANDOM FOREST FEATURE IMPORTANCE
# ============================================================

rf_model = random_forest.named_steps["model"]

feature_names_rf = (
    random_forest
    .named_steps["preprocessor"]
    .get_feature_names_out()
)

importance = rf_model.feature_importances_

feature_importance = pd.DataFrame({
    "Feature": feature_names_rf,
    "Importance": importance
})

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
).head(15)
```

```
print("\n==============================")
print("TOP RANDOM FOREST FEATURES")
print("==============================")

display(feature_importance)

plt.figure(figsize=(10, 6))

plt.barh(
    feature_importance["Feature"],
    feature_importance["Importance"]
)

plt.gca().invert_yaxis()

plt.title("Top Random Forest Feature Importances")

plt.xlabel("Importance")

plt.show()



# ============================================================
# FINAL SUMMARY
# ============================================================

print("\n")
print("================================================")
```

```
print("            PROJECT COMPLETED")
print("================================================")

print("\nDataset Shape:", df.shape)

print("\n--- Linear Regression ---")
print("MAE:", mae)
print("RMSE:", rmse)
print("R² Score:", r2)

print("\n--- Decision Tree Gini ---")
print("Accuracy:", gini_accuracy)
print("Precision:", gini_precision)
print("Recall:", gini_recall)
print("F1 Score:", gini_f1)

print("\n--- Decision Tree Entropy ---")
print("Accuracy:", entropy_accuracy)
print("Precision:", entropy_precision)
print("Recall:", entropy_recall)
print("F1 Score:", entropy_f1)

print("\n--- Random Forest ---")
print("Accuracy:", rf_accuracy)
print("Precision:", rf_precision)
print("Recall:", rf_recall)
print("F1 Score:", rf_f1)

print("\n================================================")
```

```
print("EDA + DATA CLEANING + REGRESSION +")
print("DECISION TREE + GINI + ENTROPY +")
print("RANDOM FOREST + METRICS + GRAPHS")
print("================================================")
```